Final ARC-AGI Milestone Project

Scott Schmidl
sschmidl3@gatech.edu

1 HOW DOES YOUR AGENT WORK?

My final ARC-AGI agent is rule-based and uses the Generate and Test idea. It treats each ARC Problem as a search over a finite rules bank of grid transformations and builders. For every problem, the agent loads all the training pairs and the test input, then iterates over a collection of candidate rules pulled from each milestone. Any rule that fails to reproduce every training output exactly is rejected. Any rule that passes becomes a hypothesis applied to the test input to produce the predictions. The agent collects up to three unique predictions and returns those as its final answer list.

1.1 Knowledge Base

The core knowledge is organized into a rule bank plus builders. Milestone AB, C, and D rule sets capture transformations such as symmetry, rotations, cropping, quadrant filling, staircases, hole filling, and path tracing. The builder functions and panel masks produce additional problem specific rules from the training pairs, such as treating left and right panels as a logical OR or outlining and filling shapes based on holes and borders. Shared utilities give rules an object level view of the grid such as connected components, bounding boxes, holes, borders, and enclosed regions. That lets rules reason in terms of shapes instead of individual pixels. All of these are bundled into candidate functions so the agent can simply search across operators, geometric growth, quadrants, holes and borders, and other relational patterns like path drawing and mirroring.

1.2 Control and Reasoning

For each problem, the agent extracts training pairs as a set of input and output grids and the test input grid and constructs the candidate functions by gathering first the milestone specific rule bank, then each builder, then the logical operators. It, then, filters candidates by applying each rule to every training input and checks that we return a solution and that the predicted grid equals the training output exactly. Each remaining rule is applied to the test grid and the result gets pushed into the prediction list while also removing results with no prediction and duplicates. Finally, it returns three unique predictions.

2 HOW WELL DOES YOUR AGENT PERFORM ACROSS THE SETS?

I evaluated my final agent on milestone sets B, C, and D, including both the local and hidden cases. Overall, the agent solves 84 out of 96 problems correctly, with great performance on all training pairs and strong but worse performance on the hidden cases.

2.1 Performance by Set

Table 1—Local and Hidden Performance on All Milestones

Milestone	Local	Hidden	Total	Runtime (s)
B	16/16	13/16	29/32	0.065369125
C	16/16	14/16	30/32	0.036368708
D	16/16	9/16	25/32	0.052972584
Total	48/48	36/48	84/96	0.154771750

My agent achieves 100% accuracy on the local training problems for every milestone. This suggests that the current rule bank and builders are good enough to explain all the provided cases. On the hidden cases, it achieves 36 out of 48. As the milestones get more difficult, the hidden cases move toward more abstract and relational patterns, and my rule-based design starts to show some limitations. Milestone D in particular appears to have more generalization beyond the patterns seen in the training pairs.

2.2 Runtime and Complexity

Locally, the 48 training cases run for a total of 0.154771750 seconds, while Milestone B runs in 0.06 seconds, Milestone C runs in 0.03 seconds, and Milestone D runs in 0.05 seconds. When including the extra 48 hidden cases, the runtime is 0:00:00.663533 seconds, which is well under a second.

I found these runtimes to be consistent with the time complexity. For each problem, the agent iterates over the rule bank and applies each candidate to every training grid. Most rules run in time proportional to the total number of cells they inspect. As a result, the overall complexity is on the order of $O(R \cdot A)$, where R is the number of rules and A is the total cells in the grid.

2.3 Overall Performance

Across the entire set of milestones, my agent solves 84 of 96 problems. This means that when the true transformation can be captured as a consistent rule in the current bank, the agent tends to find it and generalize reliably. When the hidden cases require either more compositional reasoning or more flexible abstractions than my rules, performance drops, even if the local cases are solved perfectly. This gap is a useful measure of how far my approach goes and where it still overfits specific patterns or colors.

3 WHAT PROBLEMS DOES YOUR AGENT SOLVE SUCCESSFULLY?

My agent performs best on problems where the transformations are clear, repeatable, stable rules across all training pairs. In these three problems my agent gets both the training and hidden cases correct. I chose one from each milestone that are significantly different in the rules that are required to solve them to show the range of behaviors my rules can capture. In each case, I describe how my agent solves it using the same Generate and Test architecture.

3.1 Milestone B: 623ea044

In problem 623ea044, each input is an empty grid with a single colored marker in a random spot. In the outputs, the single marker is expanded with diagonal arms that form a tilted T pattern centered on the marker and of the same color. Each arm extends outwards along the diagonals until it hits the grid boundary.

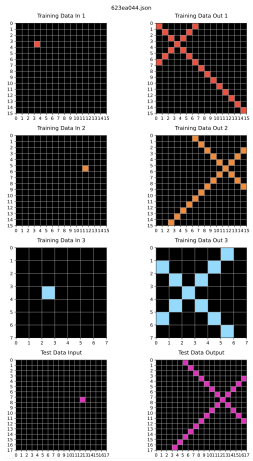


Figure 1—Expands Marker as Sideways T.

Given that this pattern is geometric and symmetric, my agent treats it as a fixed structural transformation. It detects the unique non-zero cell, then paints along the four diagonals. This is exactly the kind of regular growth rule that my rule bank is good at representing. Once the agent matches the training pairs, it is able to generalize directly to the hidden test case with a different marker position and color.

3.2 Milestone C: f8a8fe49

In problem f8a8fe49, the grid contains a larger outer frame of one color and smaller objects of another color inside that frame. The transformation essentially reflects or flips the inner objects outside the frame while preserving the outer object positions. This problem is about recognizing that there are two distinct object roles, frames versus interior and applying a reflection about an axis.

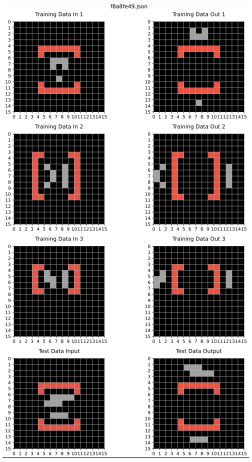


Figure 2—Flip Inside Objects Outside.

My agent handles this kind of task well because I built some of the C rules around connected components, bounding boxes, and inner versus outer relationships. Once my agent has identified the frame and inner components, it can apply a mapping from interior to exterior positions and reproduce both the training and hidden test outputs.

3.3 Milestone D: 4b6b68e5

In problem 4b6b68e5, the inputs contain multiple objects, some open and some closed, outlined with a base color and a different number of markers of different colors inside them. The outputs follow a rule that says, "for each object, if it's

open, the marker is removed and outline unchanged, but if it's closed, the interior is filled completely with the marker color that appears most frequently".

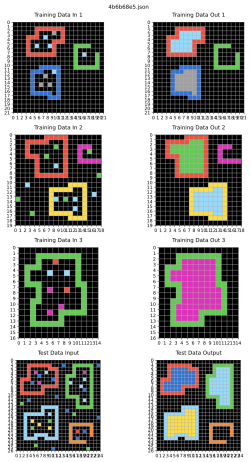


Figure 3—Object Open Remove Marker. Object Closed Fill with Majority Marker Color.

In order to solve this, my agent detects each object, determines if it's open or closed, computes a color histogram, and selects the marker with the biggest histogram bar. The rule bank for Milestone D contains those for enclosed-region detection, hole filling, and color frequency checks. Once the agent confirms the object statuses for the training pairs, the same rule is able to generalize to the hidden test cases.

4 WHERE DOES YOUR AGENT STRUGGLE?

My agent struggles most on problems that require more abstract generalizations, such as colors, or where the transformation requires complex compositions. In the following two cases, my agent can still find a rule that fits all training pairs, but that overfits to details such as layouts or colors instead of learning the deeper concept. As a result, it passes the local training pairs but fails to generalize to the hidden tests, which most likely introduce new patterns that require learning and understanding object roles and relationships.

4.1 Milestone B: d687bc17

In problem d687bc17, the behavior is to slide the markers over to their border of the same color, and markers that don't have a matching border color get removed. The training outputs show that markers can move horizontally or

vertically toward their border and stop at the boundary.

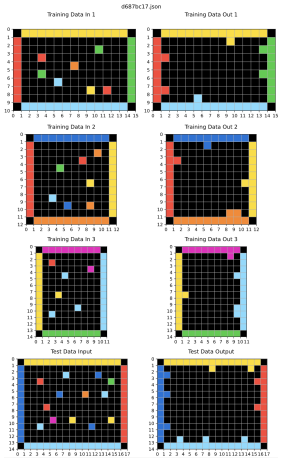


Figure 4—Marker(s) Match(es) Border Color, Slide to Border, otherwise Remove.

My agent treats the markers as objects and tries to move them towards borders, but its rule ends up relying on specific rows and columns from the training layouts instead of generalizing to "match border color, then slide over to it". When the hidden test grid changes the arrangement of markers and borders, that rule may no longer apply cleanly, so the agent's prediction diverges from the true output even though it appeared correct from the training cases.

4.2 Milestone D: 60a26a3e

In problem 60a26a3e, the core ideas are relationships and geometry, which manifests as "when two object centers line up horizontally or vertically, the output adds a connecting line of a new color but when their object centers don't line up the object is simply copied over with no connection". The four training pairs are in different locations and arrangements, but all share the same idea.

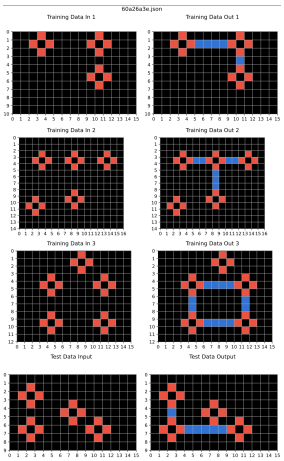


Figure 5—Objects Line Up Vertically/Horizontally, Connect with Line, otherwise Copy Over.

My agent correctly identifies the relationships between these pairs, but encodes the behavior in a way that is tied to specific patterns. In the hidden test cases, the objects are probably in a different arrangement with different colors, and my agent doesn't recognize this pattern. In this case, it must draw wrong connections or fails to connect at all, or uses the wrong color. This highlights a general weakness of the current design, i.e. it can express individual relationships but struggles to generalize to a new family of transformations that requires learning a more general rule.

5 HOW WOULD YOU CHARACTERIZE YOUR OVERALL APPROACH TO DESIGNING YOUR AGENT?

My agent follows a consistent rule-based design approach. The rule bank itself grew with each milestone and was updated as I encountered new kinds of problems. I didn't throw out the architecture, instead I treated each milestone as an opportunity to expand and refine the bank of transformations while keeping the same Generate and Test control loop.

5.1 Milestones A and B

For Milestones A and B, I focused on simple, concrete transformations. I built rules for overlap, cropping, rotations, mirroring, hollow rectangles, dilation, and a recursive spiral patterns. I also added color heuristics and logical operators. The main idea was to capture the most common transformations I saw in the

examples, i.e. symmetric patterns, cropping out specific clusters, and recoloring. The architecture of using proposed rules and testing them against all training pairs, then applying successful ones to the test grid seemed like it could cause complexity issues, but the agent stayed quite quick.

5.2 Milestone C

For Milestone C, I extended this design instead of replacing it. I added more structural and object-centric rules, such as building quadrants and tilings, transposing grids, folding and mirroring patterns, creating staircases, and reorganizing colors by frequency. I also leaned more on shared utilities to detect connected components, bounding boxes, holes, and borders. The high-level pattern was the same, i.e. grow the rule bank to cover more geometry and color roles, but keep the same Generate and Test loop and the same requirement that rules must pass all training pairs exactly.

5.3 Milestone D

In Milestone D, I continued expanding the rule bank with those aimed at more abstract visuospatial reasoning. I added operators for path tracing, logical overlap, mirroring, enclosed regions, block counting, color pairs, and mosaics. These rules use the same utilities but think more in terms of relationships between objects, not just individual shapes. Again, the agent continues to treat each problem as constrained hypothesis testing over a finite bank of rules.

5.4 Full Pattern

Overall, my approach is best described as an expanding bank of heuristics and operators running inside a Generate and Test loop. When I saw new problems that my agent missed, I responded by adding or generalizing rules rather than redesigning the core idea. The control flow, knowledge organization, and basic search strategy have been constant. What has changed over time is the breadth and depth of the rule bank, moving from simple pixel level patterns to more geometric, object-centric, and relational transformations.

6 HOW SIMILAR DO YOU FEEL YOUR FINAL AGENT IS TO HOW YOU WOULD APPROACH THE TEST?

6.1 Similarities

There are several places where my agent's strategy overlaps with how I naturally solve ARC-AGI problems. As a human, I usually start by identifying the structure of the grids, looking for similar patterns, and then deciding on a transformation that explains the training pairs. My agent does something similar in a slightly different way. It assumes that each problem can be solved by a single, consistent rule, then searches the finite rule bank for a transformation that reproduces all training outputs exactly. In that sense, both of us are doing a form of Generate and Test that uses the samples as constraints on which explanations are acceptable.

We also rely on the same kinds of patterns. When I look at problems like cropping, overlap, quadrants, staircases, or path tracing, I recognize them as whole ideas such as cropping out non-zeros sections or connecting markers with a line. The rule bank encodes those same ideas as operators. Once the agent finds a rule that fits the training pairs, it generalizes that pattern to the test input in a way that is quite close to how I would apply the same idea by hand.

6.2 Differences

The biggest difference is flexibility. I can reframe a problem when my first guess fails. If I initially think a problem is about rotation but then notice it is actually about color roles or inside versus outside, I can change my strategy on the fly. My agent doesn't do that. Once a rule fails on even one training pair, it is discarded entirely, and there is no process to adapt, compose, or repair rules. The agent doesn't do much meta-reasoning about its own failure.

Another difference is representation. I can naturally treat some grids as having a multiple-layer structure, such as a grid of grids or objects that carry roles like markers, border, or a center. The agent only has the representations that I explicitly encoded such as connected components and bounding boxes. That is enough for some problems, but it breaks down on tasks that require better relational reasoning between multiple objects or a hierarchical view of the grid. This shows up clearly in some Milestone D failures, where I can see what the hidden case is doing conceptually, but the existing rules still overfit to the patterns in the

training pairs.

6.3 Overall Comparison

Overall, I would characterize my agent as being similar to my own approach in spirit but not in depth. It shares the idea that these problems can be solved by recognizing structure and applying right operator, and it uses the training pairs to find which operators are valid. At the same time, it is much more brittle than my own reasoning. It doesn't discover new abstractions, doesn't easily combine multiple steps, and doesn't adjust its strategy when the first explanation fails. In that sense, it behaves more like a careful person who has memorized many techniques and applies them consistently, rather than a fully flexible human who can improvise new strategies when needed.